Development of a Web-Based Self-Driving Car Rental Support Platform Utilizing Java Servlet and SQL Server

Author 1, Author 2, and Author 3

FPT University, Da Nang, Vietnam
{auth1, auth2, auth3}@fpt.edu.vn

Abstract. The rise of the sharing economy has drastically altered consumer vehicle utilization habits. This paper presents the design and implementation of a web-based self-driving car rental support platform built on a Model-View-Controller (MVC) architecture using Java Servlet, JSP, and SQL Server. The platform aims to digitalize conventional rental operations by facilitating vehicle fleet status tracking, automated booking workflows, and electronic rental agreement processing. Furthermore, a payment gateway mechanism is seamlessly simulated using the VNPAY Sandbox environment to handle deposit collection and automated refunds. Experimental evaluations indicate that the platform significantly boosts operational transparency and transaction efficiency across stakeholders.

Keywords: Self-driving car rental · Java Servlet · VNPAY API · Fleet control · E-commerce architecture.

1 Introduction

Modern web ecosystems have facilitated massive peer-to-peer sharing capabilities, notably within the personal transportation sector. Traditional rental configurations, however, still rely heavily on fractured, non-centralized data points, which creates operational vulnerabilities regarding car allocation, precise battery tracking, and security deposits.

To address these pain points, this research implements an enterprise-ready web-based application built upon Java Servlet, JSP, and SQL Server backend configurations[cite: 80]. The structural scope encompasses automated contract tracking, fleet status parameters (including battery level evaluation at handover), and integrated online billing solutions via a standard VNPAY Sandbox framework[cite: 81, 82].

2 System Architecture and Structural Stakeholders

Analysis of system interaction workflows identifies 5 primary functional stakeholders (actors) interacting with the underlying architecture:

1. **Guest:** Unauthenticated entities authorized to browse active electric vehicle catalogs, verify dynamic pricing matrices, view maps of charging station networks, and look through general guidelines.
2. **Renter:** Registered users possessing full operational permissions to reserve cars, schedule routes, execute deposit transactions, handle vehicle pickups/returns via digital forms, and evaluate post-trip metrics.
3. **Car Owner:** Fleet supply partners who register vehicles into the active inventory, process incoming rental reservations, and monitor financial revenue statistics.
4. **Administrator (Admin):** Global managers who oversee entire platform configurations, approve listings, monitor monetary flows, and execute automated refund requests.
5. **Customer Services (CS):** Support entities responsible for Know-Your-Customer (KYC) image processing, verifying driver's licenses, and handling operational disputes.

3 Functional Use Case Mapping

The core operational logic of the platform is modularly partitioned into 35 unique Use Cases (UC01–UC35) to manage systemic actions smoothly:

3.1 Identity Verification and Authentication Block

Managing foundational access vectors encompasses secure system registration (UC01), Google OAuth single sign-on integration (UC02), and strict data submission for identity validation (UC04). Users must upload official national identity cards and verified driver's credentials to satisfy platform safety regulations.

3.2 Renter Navigation and Booking Ecosystem

Unauthenticated users interface with basic search criteria and multi-variable filtering matrices (UC09) to sort listings by price parameters, location, and vehicle seating capacity[cite: 94]. Upon moving to a formal reservation state, verified Renters initialize online checkout and payment components (UC14) utilizing external APIs to fulfill essential deposit obligations safely.

3.3 System Control Matrix

Table 1 outlines the exact structural mapping of critical operational parameters across key modules of the system.

Table 1. Platform Functional Segmentation and Stakeholder Mapping

Functional Group	Core System Logic	Interacting Actors
Authentication Block	Registration, Session Login, KYC Verification	All Users [?,?]
Rental Lifecycle	Car Search, VNPAY Deposits, Pickup/Return Handover	Renter, Owner [?,?]
System Governance	Fleet Control, Audit Logs, Deposit Allocation	Admin, CS Staff [?,?,?]

4 Technical Architecture and Implementation

The software infrastructure employs an enterprise-tier Model-View-Controller (MVC) architectural design pattern to optimize separation of concerns:

- **Presentation Tier:** Constructed with robust JavaServer Pages (JSP), HTML5, CSS3, and native JavaScript to render fluid user dashboards across device breakpoints[cite: 80].
- **Application Controller Tier:** Managed entirely by Java Servlets acting as centralized endpoints to ingest incoming HTTP payloads, parse parameter states, and handle database lookups[cite: 80].
- **Data Storage Tier:** Anchored on relational SQL Server clusters ensuring optimal query routing, relational mapping rules, and absolute transactional consistency[cite: 80].

5 Conclusion and Future Research Direction

This research demonstrates the viability of a secure, transparent, and scalable self-driving car rental framework built with an MVC architecture[cite: 80]. By combining strict driver KYC verification loops with automated, electronic transaction workflows, the system mitigates financial and physical asset risks for vehicle owners[cite: 81, 92]. Future development phases will integrate predictive Machine Learning algorithms to offer dynamic vehicle pricing and automated asset tracking through IoT hardware endpoints.

References